When Does Compressed KV Start Lying?

Token-Level Drift in KV Cache Compression

ExactKV Project
Research release (git tag `v-release`)

June 2026

Quantitative values from `reports/scale_7b/raw.json`, `reports/evidence_plus/raw.json`, `reports/external_panels/` (including `v30/`), and `reports/public_release/leaderboard_final.json`. Public cite: ***ExactKV research release*** *· git tag `v-release`. Panel-batch labels like `v30/` are internal artifact names, not separate public versions. Claim boundaries: `docs/CLAIM_BOUNDARIES.md`.*

Abstract

Lossy KV-cache compression is widely deployed, but most evaluations report aggregate quality and stop there. ExactKV is a compressor-agnostic **crash-test framework** that measures first-divergence index, draft acceptance, and `exactkv_failure` under verifier-mediated semantics.

Across **8,132 GPU cells** on Llama-3.1-8B and Mistral-7B (Appendix benchmark card), all panels report `exactkv_failures=0`[1]. Four main findings:

1. **Task type dominates.** `int4_sim`: 6% (MBPP code) → 11% (BFCL short) → 50% (BFCL long-gen) → **90%** (HF LongBench). H2O-style eviction at 75% kept: **100%** divergence on LongBench.
2. **Generation length scales within-task.** On BFCL, `int4_sim` rises from 9% (mnt=16) to 62% (mnt=256), a 7× increase.
3. **Three distinct failure modes.** Top-k logit autopsy (1,103 divergent cells): near-tie noise (int8, rank 2.4, fdi=22). distribution shift (int4_sim, rank 3.5, fdi=8). attention destruction (H2O-style, rank 6.7, fdi=1). Three forensic case studies confirm each.
4. **100% downstream validity preserved.** Despite 50% token drift, ExactKV preserves all 106/106 full-KV valid BFCL tool calls (both models, all four task categories).

`int6_sim` (6-bit per-tensor) and `int4_per_vec_sim` (KIVI/KVQuant-style per-vector INT4, 2.06× int8 error) are GPU-validated non-catastrophic on both Mistral-7B and Llama-3.1-8B (v3.0, 1,568 cells total, `exactkv_failures=0`): both show 0% divergence on BFCL and MBPP. `int6_sim` reaches 37–47% and `int4_per_vec_sim` 56–57% on HF LongBench, per-vector granularity helps on structured tasks, while bit-width still matters at 8K LongBench context. Results are **not** benchmark scores, production claims, or a reproduction of VeriCache [8].

1 Introduction

Lossy KV-cache compression is widely deployed as a transparent optimization for LLM inference. Under greedy decoding, a single perturbed logit can flip the argmax, after which trajectories diverge.

[1] Six built-in compressor classes: `noop`, `int8`, `int6_sim`, `int4_per_vec_sim`, `int4_sim`, H2O-style eviction (`h2o_sim`). External adapters (`kivi_offline`, `kivi_offline_r32`, SnapKV) evaluated separately (Table 11, Section 14.1).

Aggregate metrics (perplexity, LongBench, task accuracy) average over this divergence and hide *where* it begins, *how severe* it is, and *which compressor design choices* drive it.

ExactKV reframes KV-cache evaluation around a single diagnostic question: **at what generated token does the compressed-cache path first stop matching full-KV greedy decoding?** We answer this with a compressor-agnostic crash-test framework that runs a draft/verify/commit loop and records first-divergence index, draft acceptance, verifier agreement, and exactness failures per cell.

Contributions:

1. A **compressor-agnostic crash-test framework** with leaderboard-style reporting for token-level drift, first-divergence index, acceptance rate, and exactness failures across compressors and models (§3–5).
2. **8,132 GPU cells** across Llama-3.1-8B and Mistral-7B, five benchmark families, six built-in compressor classes, with `exactkv_failures=0` throughout (§6).
3. Empirical evidence that **task type dominates drift** (6% code → 90% reading for `int4_sim`) and **generation length scales it** (9% → 62% on BFCL, 7×) (§6.4–6.12).
4. A **logit autopsy** over 1,103 divergent cells identifying three mechanistically distinct failure modes: near-tie noise (int8), distribution shift (int4_sim), and attention destruction (H2O eviction) (§10, §15).
5. **Downstream validity measurement**: despite 50% token drift, all 106/106 full-KV valid BFCL tool calls are preserved (§6.11).
6. **GPU validation** of `int6_sim` and `int4_per_vec_sim` confirming non-catastrophic drift on structured tasks. per-vector granularity helps on BFCL/MBPP while bit-width still matters at 8K LongBench context (Sections 13–14, Table 25).

1.1 Shared problem framing with VeriCache

VeriCache [8] and ExactKV share the observation that lossy KV may look acceptable on short or aggregate metrics but **diverges more as decoding continues**, and catastrophic for code and tool-calling. VeriCache uses compressed KV to **draft** and full KV to **verify/correct**, guaranteeing **identical greedy output** to full-KV inference.

1.2 Where ExactKV diverges from VeriCache

VeriCache is a **serving/system** paper. Its contribution is not merely "draft then verify." It makes that loop practical by: (1) keeping compressed KV on GPU, (2) keeping full KV in CPU/storage, loading only for verification, (3) **cross-resource staggering** (HBM-bound draft vs interconnect-bound verify), (4) reporting serving throughput (up to ∼4× vs full-KV in their evaluation) with identical outputs.

ExactKV is an **evaluation crash-test**. It measures first divergence, acceptance, and exactness failures, *not* deployment throughput. It does *not* reproduce VeriCache scheduling or memory tiering.

1.3 Main Findings at a Glance

#	Finding	Key number
1	Task type dominates drift	int4_sim: MBPP 6% → BFCL 11% → BFCL long 50% → LongBench 90%
2	Generation length scales drift	int4_sim mnt=16: 9% → mnt=256: 62% (7×)
3	Eviction > quantization	H2O-style 75% kept → 100% LongBench divergence
4	Three distinct failure modes	int8: near-tie (rank 2.4, fdi=22). int4_sim: shift (rank 3.5, fdi=8). H2O: destruction (rank 6.7, fdi=1)
5	100% downstream validity	ExactKV preserves 106/106 valid BFCL tool calls despite 50% drift
6	Zero correctness failures	`exactkv_failures=0` across all 8,132 GPU cells

Table 1: Main findings summary.

2 Definitions

2.1 Reference paths

- **Full-KV reference:** greedy generation with uncompressed KV.
- **Lossy path:** compressed-KV greedy without verifier.
- **ExactKV path:** verifier-mediated draft/verify/commit (`ExactKVGenerator`).

2.2 Metrics (formal)

Table 2: Core metrics (formal definitions).

Metric	Definition
`first_divergence_index`	First position where the *lossy* token $\neq$ full-KV greedy (before correction).
`acceptance_rate`	`total_accepted / total_drafted` across verification rounds.
`exactkv_failure`	Final ExactKV output $\neq$ full-KV greedy (verifier/commit failure).
`divergence_rate`	Fraction of cells with lossy-path divergence.
`divergence_stability`	$1 -$ `divergence_rate` (complement).
`compression_ratio`	Stored tensor byte ratio only (not active GPU memory).

Corrections vs rejections: `total_rejected` counts rejected *draft tokens*, `total_corrections` counts verification *rounds* with a correction. For `int4_sim`: 1284/1502 accepted, 218 rejected tokens, **116 correction rounds**.

2.3 Algorithm

```
Prefill -> FullKVState + CompressedKVState
while generated < max_new_tokens:
  DRAFT from compressed KV (up to draft_len tokens)
  VERIFY each token vs full-KV greedy
  COMMIT accepted prefix, correct on mismatch
  ALIGN compressed state from authoritative full KV
return ExactKV output_ids
```

Greedy decoding only in this release.

3 Method

Each **cell** is (model, compressor, prompt, `max_new_tokens`) with `draft_len=4`, `seed=0`. Three modes per cell: **full**, **lossy**, **exactkv**. Leaderboard score: $0.35\,a + 0.25\,v + 0.20(1 - \hat{d}) + 0.10(1 - f) + 0.10\,s$.

4 Experimental Setup

Table 3: Headline release panel (`reports/scale_7b/raw.json`).

Parameter	**Value**
Panel ID	`phase_a_unified_panel`
Total cells	**1500** $= 2 \times 5 \times 50 \times 3$
Models	Llama-3.1-8B (750), Mistral-7B (750)
Stack	torch 2.8.0+cu128, transformers 5.12.1, ExactKV research release (tag `v-release`)
`draft_len`	4 `max_new_tokens` $\in \{4, 8, 16\}$
Decoding	Greedy only
Prompts	50 variants from 4 stress templates (capital, math, JSON, code)
Categories	capital_france 390, simple_math 390, json_tool 360, code_fn 360
Execution	Sequential per model, `deterministic_mode=false`

Validated compressors: `noop`, `int8`, `int4_sim`. Real KIVI/KVQuant/SnapKV/TurboQuant are not in the headline panel. The raw JSON also contains proxy/probe slots (`spectralquant`, `shard`) excluded from all analysis. see Limitations (§22).

1,560 GPU cells total: 216 Llama-3.1-8B cells across bundled LongBench/RULER/BFCL/HumanEval pilots. 144 MBPP cells across both models (`mbpp_gpu_raw.json`). 1,200 BFCL export-50 tool-call drift cells across both models, `exactkv_failures=0` throughout. Not official benchmark scores.

5 Results: Validated Compressors

Table 4: built-in compressors with direct ExactKV integration (`noop`, `int8`, `int4_sim`). Aggregates from `compressor_summary` in `reports/scale_7b/raw.json` (300 cells each: 150 per model).

Compressor	**Cells**	**Accept.**	**Div.**†	**Stab.**	**1st div.**§	**Fail.**
`noop`	300	1.000	0.00	1.00	n/a	0.00
`int8`	300	1.000	0.00	1.00	n/a	0.00
`int4_sim`	300	0.851	0.52	0.48	2.0	0.00

Table 4: Built-in compressors from `compressor_summary`. †Lossy-path divergence fraction. §Mean over divergent cells only. Histogram for `int4_sim`: index 1→78, 3→39, 5→24, 8→13 cells.

`int8`/`noop`: no lossy divergence. `int4_sim`: drifts in 52% of cells yet `exactkv_failure=0` because the verifier corrects.

Leaderboard-style scores (formula: $0.35a + 0.25v + 0.20(1 - \hat{d}) + 0.10(1 - f) + 0.10s$, source: `reports/public_release/leaderboard_final.json`):

Compressor	**Model**	**Score**	**Accept.**	**Div. rate**[†]	**Stab.**	**Cells**
`noop`	Llama-3.1-8B	1.000	1.000	0.000	1.000	150
`int8`	Llama-3.1-8B	1.000	1.000	0.000	1.000	150
`noop`	Mistral-7B	1.000	1.000	0.000	1.000	150
`int8`	Mistral-7B	0.983	1.000	0.000	0.827	150
`int4_sim`	Llama-3.1-8B	0.859	0.852	0.520	0.480	150
`int4_sim`	Mistral-7B	0.851	0.837	0.507	0.493	150

Table 5: Leaderboard-style scores for validated built-in compressors. [†]Div. rate = raw lossy-path token divergence fraction (same metric as Table 4). `int8` Mistral: divergence_rate=0.000 (zero cells diverge). `Stab.`=0.827 is the **leaderboard stability subscore** from `leaderboard_final.json`, which is distinct from the formal metric divergence_stability = 1−divergence_rate (Table 2). under the formal metric, `int8` Mistral divergence_stability=1.000. `spectralquant` (MOCK→`int4_sim`) and `shard` (PROBE_ONLY) excluded from ranked table.

5.1 Metric interpretation

Divergence rate and acceptance rate measure *different* failure modes. Low lossy-path divergence does not imply high verifier acceptance (and vice versa). Example: `int4_sim` at `max_new_tokens=16` can show first divergence at index 8 with acceptance 0.94, how late drift occurs matters.

5.2 Evidence-plus long-context panel

Source: `reports/evidence_plus/raw.json` (RunPod A5000, June 2026). 144 cells = $2 \times 6 \times 2 \times 3 \times 2$, prefill buckets 512/1024, `max_new_tokens` $\in \{16, 32\}$, built-in compressors only. By context

Compressor	Cells	Accept.	Div. rate	Fail.
`noop`	48	1.000	0.00	0.00
`int8`	48	1.000	0.00	0.00
`int4_sim`	48	0.985	0.31	0.00

Table 6: Evidence-plus aggregates. `int4_sim`: Mistral 41.7%, Llama 20.8% divergence (24 cells each).

bucket (all compressors): 512→11.1% divergence, 1024→9.7%. Diagnostic timing: mean 3854 ms, p90 5178 ms per cell (not serving throughput).

5.3 External benchmark smoke panels

Source: `analysis_pack.json` (216 Llama-only), `mbpp_gpu_raw.json` (144 cells). `exactkv_failures=0` on all panels. 216-cell subset: `noop`/`int8`: 0%. 15 divergent cells (`int4_sim` only).

Panel	Source	Model	Ctx (K)	MNT	Cells	Div.	Acc.	Fail.	P90 ms
LongBench	pilot	Llama-3.1-8B	2,4	16,32	72	0.083	0.994	0	8781
RULER 2K–4K	pilot	Llama-3.1-8B	2,4	16,32	48	0.083	0.993	0	8793
RULER 8K	pilot	Llama-3.1-8B	8	16,32	24	0.083	0.993	0	13640
BFCL	pilot (4)	Llama-3.1-8B	1,2	16,32	48	0.063	0.999	0	6342
HumanEval	pilot (4)	Llama-3.1-8B	1,2	32	24	0.000	1.000	0	6392

Table 7: Llama-3.1-8B external smoke panels (216 cells, `noop/int8/int4_sim`). Drift metrics. not official scores.

Panel	Source	Models	Ctx (K)	MNT	Cells	Div.	Acc.	Fail.	P90 ms
MBPP	pilot (6)	Llama+Mistral	0.5,1	16,32	144	0.021	0.999	0	5121

Table 8: MBPP code-drift smoke (144 cells, `noop/int8/int4_sim`). Not pass@1. no test execution.

Compressor	Model	Cells	Div. rate	CI_{95} low	CI_{95} high
`noop`	Llama-3.1-8B	200	0.000	0.000	0.019
`noop`	Mistral-7B	200	0.000	0.000	0.019
`int8`	Llama-3.1-8B	200	0.000	0.000	0.019
`int8`	Mistral-7B	200	0.000	0.000	0.019
`int4_sim`	Llama-3.1-8B	200	0.055	0.031	0.096
`int4_sim`	Mistral-7B	200	0.170	0.124	0.228

Table 9: BFCL export-50 tool-call drift panel (1,200 cells, both models, 50 prompts). `exactkv_failures=0`. Source: `bfcl_export_50_raw.json`. Drift measurement only (not JSON-completeness). `max_new_tokens`$\in \{16, 32\}$. Mistral-7B `int4_sim` divergence (17.0%) is 3× Llama-3.1-8B (5.5%).

Notable: LongBench `int4_sim` 25%. RULER 8192 `niah_single` 33%. BFCL pilot `int4_sim` 18.75%. BFCL export-50 `int4_sim` 11.3% (Llama 5.5%, Mistral 17.0%). HumanEval zero drift. MBPP 3/144 divergent. `int8/noop`: 0% divergence on all panels.

Panel	n	Div.	Rate	CI_{95} low	CI_{95} high
LongBench pilot	72	6	8.3%	3.9%	17.0%
RULER 2K–4K	48	4	8.3%	3.3%	19.7%
RULER 8K	24	2	8.3%	2.3%	25.8%
BFCL pilot	48	3	6.3%	2.2%	16.8%
HumanEval pilot	24	0	0.0%	0.0%	14.2%
MBPP pilot	144	3	2.1%	0.7%	6.0%
BFCL exp-50 `int4_sim` Llama	200	11	5.5%	3.1%	9.6%
BFCL exp-50 `int4_sim` Mistral	200	34	17.0%	12.4%	22.8%
BFCL exp-50 `int8/noop`	800	0	0.0%	0.0%	0.5%
All 216 ext. cells	216	15	6.9%	4.3%	11.1%

Table 10: Wilson 95% CIs on divergence rate (panel-level). Source: `scripts/build_external_analysis_pack.py`. "All 216 ext. cells" refers to the **initial** LongBench/RULER/BFCL/HumanEval pilot panel (first workflow, Llama-only). the full 1,560-cell external smoke total includes MBPP and BFCL export-50.

Overall acceptance (initial 216 ext. pilot cells): Full-acceptance rate 93.7%, Wilson 95% CI [90.5%, 96.8%]. Source: `analysis_pack.json` → `totals.acceptance_full_rate_ci95`.

5.4 KIVI offline compressor panel, real HF results (640 cells)

Source: `kivi_longbench_hf_raw.json` (320 cells) + `kivi_mbpp_hf_raw.json` (320 cells). RunPod RTX A5000, June 2026. `exactkv_failures=0` on all 640 cells.

Compressor	LB div.	LB acc.	MBPP div.	MBPP acc.	Fail.
`noop`	0.0%	1.000	0.0%	1.000	0
`int8`	13.8%	0.994	0.0%	1.000	0
`int4_sim`	91.2%	0.818	5.0%	0.995	0
`kivi_offline`	**100.0%**	**0.004**	**100.0%**	**0.001**	**0**

Table 11: KIVI offline compressor panel (80 cells each $\times$ 4 compressors $\times$ 2 datasets). `kivi_offline` produced catastrophic **adapter-level** drift in this ExactKV integration (near-zero acceptance). Because this is an offline simulate-path adapter rather than KIVI production CUDA/Triton serving, these results diagnose the adapter/integration path, *not* the KIVI algorithm as deployed. ExactKV detected and corrected all cells (`exactkv_failures=0`). Real HF `int4_sim` drift (LB: 91.2%) greatly exceeds bundled pilot (20.8%).

6 ExactKV-HF-LongBench Drift Panel (v2.6, Complete)

Status: Complete. 720 cells, both models, `exactkv_failures=0`.

Model	Compressor	n	Div.	Rate	CI_{95}	Mean acc.	EKV fail
Llama-3.1-8B	noop	120	0	0.0%	[0.0%, 3.1%]	1.000	0
Llama-3.1-8B	int8	120	33	27.5%	[20.3%, 36.1%]	0.988	0
Llama-3.1-8B	int4_sim	120	110	**91.7%**	[85.3%, 95.4%]	0.825	0
Mistral-7B	noop	120	0	0.0%	[0.0%, 3.1%]	1.000	0
Mistral-7B	int8	120	26	21.7%	[15.2%, 29.9%]	0.988	0
Mistral-7B	int4_sim	120	107	**89.2%**	[82.3%, 93.6%]	0.861	0
All (int4_sim)	all	**240**	**217**	**90.4%**	**[86.1%, 93.5%]**	0.843	**0**

Table 12: ExactKV-HF-LongBench v2.6 (720 cells, 10 real HF subsets, contexts 2K/4K/8K, `mnt`=32/64, both models). int4_sim: **90.4%** divergence on long-context open-text tasks, highest across all ExactKV panels. int8 shows 24.6%, unlike its 0% on BFCL/MBPP/RULER. `exactkv_failures=0`. Median first-div index: 4 (Llama) / 6 (Mistral).

Key finding (task-type sensitivity): Real HF LongBench open-text reading/summarization at 2K–8K prefill produces **very high int4_sim divergence** (90.4%), the highest seen across all ExactKV panels. Even int8 shows 24.6% divergence on LongBench tasks, unlike its 0% on BFCL, MBPP, and RULER. Divergence occurs very early (median token 4–6 out of 32–64 generated). `exactkv_failures=0` across all 720 cells.

Table 13 shows per-subset int4_sim divergence. `lcc` (code completion) is notably lower for Mistral (42%), consistent with code tasks being more stable under int4_sim across panels.

Subset	Type	Llama int4_sim	Mistral int4_sim
2wikimqa	multi-hop QA	12/12 (100%)	12/12 (100%)
gov_report	summarization	11/12 (92%)	12/12 (100%)
hotpotqa	multi-hop QA	12/12 (100%)	12/12 (100%)
lcc	**code completion**	**11/12 (92%)**	**5/12 (42%)**
multifieldqa_en	multi-field QA	6/12 (50%)	10/12 (83%)
narrativeqa	reading comp.	12/12 (100%)	12/12 (100%)
passage_retr.	doc. retrieval	12/12 (100%)	12/12 (100%)
qasper	academic QA	12/12 (100%)	12/12 (100%)
samsum	dialogue summ.	10/12 (83%)	12/12 (100%)
trec	classification	12/12 (100%)	8/12 (67%)

Table 13: Per-subset int4_sim divergence in HF LongBench v2.6 (2K/4K/8K context, `mnt`=32/64). `lcc` (code completion) shows lower divergence for Mistral (42%) vs. reading-comprehension tasks (100%), consistent with code-task stability across all ExactKV panels. `exactkv_failures=0` for all 720 cells.

7 BFCL Tool-Call Validity Panel (v2.7, Complete, 1,200 cells)

The 48-cell BFCL pilot used `max_new_tokens` $\in$ `{16, 32}`, too short for complete JSON tool calls. A dedicated 1,200-cell validity panel (`max_new_tokens=128/256`) was completed with both models.

Compressor	Model	n	Div.	Rate	EKV fail
noop	Llama	200	0	0.0%	0
noop	Mistral	200	0	0.0%	0
int8	Llama	200	0	0.0%	0
int8	Mistral	200	3	1.5%	0
int4_sim	Llama	200	90	**45.0%**	0
int4_sim	Mistral	200	111	**55.5%**	0
All (1,200)	both	1,200	204	**17.0%**	**0**

Table 14: BFCL tool-call validity panel v2.7 (1,200 cells, mnt=128/256, both models). `int4_sim` diverges 45.0% on Llama and 55.5% on Mistral with long generation, vs 11% at mnt=16/32, confirming generation length as a primary driver. `int8` is near-zero. `exactkv_failures=0` throughout. Artifact: `bfcl_validity_v27_merged_raw.json`.

8 H2O-Style Token-Eviction Panel (v2.8, Complete, 800 cells)

Status: Complete. 800 cells, 5 eviction variants, both models, `exactkv_failures=0`. Artifact: `h2o_v28_merged_raw.json`.

Eviction mechanism: `h2o_sim_75`, `h2o_sim`, and `h2o_sim_25` keep approximately 75%, 50%, and 25% of prefill tokens using a simplified heavy-hitter-style eviction heuristic (cumulative attention score across heads). Lowest-scoring tokens are evicted after the full prefill. retained tokens include attention sinks (positions 0–3) plus the highest-attention tokens by recency/score. This approximates the H2O [9] eviction policy but omits the original online streaming update and GPU-efficient sparse-attention kernel. Results characterize the eviction compressor *class*, not the published H2O system.

Key result: Even mild eviction (75% of tokens kept) causes **100% divergence** on LongBench reading tasks for both models. The H2O-style simulation reaches mean lossy rank 6.5–6.7 at divergence (vs. rank 3.5 for `int4_sim`), confirming attention destruction as a qualitatively distinct failure mode. `exactkv_failures=0` across all 800 cells.

9 Cross-Panel Divergence Analysis: Task-Type Sensitivity

Table 15 summarises int4_sim, int8, and noop divergence rates across all completed ExactKV panels, revealing task type as the dominant factor.

Panel	Task	Ctx	mnt	Models	noop	int8	int4_sim
Headline (v1)	mixed	∼500	16/32	L+M	0.0%	0.0%	**51.3%**
Evidence-plus (v2)	mixed	512/1024	16	L+M	0.0%	0.0%	**31.2%**
MBPP code-drift	code	512/1024	16/32	L+M	0.0%	0.0%	**6.2%**
BFCL export-50	tool-call	1K–2K	16/32	L+M	0.0%	0.0%	**11.2%**
BFCL validity v2.7	tool-call	1K–2K	128/256	L+M	0.0%	0.8%	**50.3%**
HF LongBench v2.6	reading/summ.	2K–8K	32/64	L+M	0.0%	24.6%	**90.4%**

Table 15: Cross-panel int4_sim/int8/noop divergence rates. noop=0% throughout confirms correct baseline. Task type dominates: code (6%) < tool-call short-gen (11%) < tool-call long-gen (50.3%) < reading (90%). int8 is zero on BFCL export-50/MBPP/RULER, near-zero on BFCL validity (0.8%), and substantial only on LongBench (24.6%). Generation length also matters (BFCL 11%→50% as mnt 16→256). `exactkv_failures=0` across all panels. L=Llama-3.1-8B, M=Mistral-7B.

10 Mechanistic Analysis: Divergence Autopsy

The cross-panel task-type hierarchy is supported by a token-level mechanistic analysis. ExactKV stored top-5 full-KV and lossy logit distributions at each divergence point (`--store-top-k-logits`), enabling a logit autopsy across 1,103 divergent cells from v2.6, v2.7, and v2.8 panels.

Compressor	n div	top-1 flip	near-tie	mean margin	mean rank	med. fdi
int8	62	69%	**66%**	0.116	2.4	22
int4_sim	563	83%	26%	0.305	3.5	8
h2o_sim_75	160	**100%**	0%	0.545	6.5	1
h2o_sim	160	**100%**	0%	0.536	6.7	1
h2o_sim_25	158	**100%**	1%	0.551	6.5	1

Table 16: Logit autopsy across 1,103 divergent cells. near-tie: full-KV margin < 0.05. fdi = first-divergence index. Three distinct failure modes: (1) *near-tie noise* (int8), (2) *distribution shift* (int4_sim), (3) *attention destruction* (H2O-style). `exactkv_failures=0` across all 1,103 cells.

Three qualitatively distinct failure modes emerge:

(1) Near-tie noise (int8): 66% of divergences are near-ties (margin < 0.05). Lossy mean rank 2.4. late divergence (fdi=22). Small 8-bit noise flips only closely-contested decisions, explaining why int8 diverges on LongBench (uncertain outputs) but not structured tasks.

(2) Distribution shift (int4_sim): 83% top-1 flip, 26% near-tie, mean margin 0.305. The full-KV model is moderately confident, yet int4_sim selects ∼3rd–4th ranked token (mean rank

3.5), diverging at median token 8. Compression biases the activation toward plausible but incorrect alternatives.

(3) Attention destruction (H2O-style): 100% top-1 flip, 0% near-tie, mean margin 0.54, median fdi=1. Eviction eliminates context anchors from the very first generated token. the lossy model selects tokens ranked 6th–7th in the full-KV distribution.

These three modes directly explain the task-type hierarchy: near-tie noise only surfaces on long-context uncertain tasks (LongBench). distribution shift affects all tasks but worsens with longer generation. attention destruction is immediate regardless of task. `exactkv_failures=0` across all 1,103 divergent cells.

11 Downstream Task-Impact Metrics

Token-level drift is ExactKV's primary metric, but downstream task output validity is the ultimate concern. We report tool-call validity preservation from BFCL v2.7.

Compressor	**Model**	**n**	**Drift**	**Full-KV valid**	**ExactKV valid**	**Preserved**
noop	Llama	200	0.0%	63/200 (31.5%)	63/200 (31.5%)	**100%**
noop	Mistral	200	0.0%	43/200 (21.5%)	43/200 (21.5%)	**100%**
int8	Llama	200	0.0%	63/200 (31.5%)	63/200 (31.5%)	**100%**
int8	Mistral	200	1.5%	43/200 (21.5%)	43/200 (21.5%)	**100%**
int4_sim	Llama	200	45.0%	63/200 (31.5%)	63/200 (31.5%)	**100%**
int4_sim	Mistral	200	55.5%	43/200 (21.5%)	43/200 (21.5%)	**100%**

Table 17: BFCL tool-call validity preservation (v2.7, 1,200 cells). Despite 45–55% token-level drift for `int4_sim`, ExactKV preserves **100%** of full-KV valid tool calls for both models. *Note: "preservation" means ExactKV matches full-KV output, not that full-KV itself always produces valid calls. The full-KV baseline achieves only 26.5% (106/400) valid tool calls on this panel.*

Category	**n**	**Drift**	**Full-KV valid**	**ExactKV valid**	**Preserved**
simple	104	49.0%	38/104 (37%)	38/104 (37%)	**100%**
parallel	104	54.8%	31/104 (30%)	31/104 (30%)	**100%**
multi_turn	104	59.6%	24/104 (23%)	24/104 (23%)	**100%**
ast_eval	88	35.2%	13/88 (15%)	13/88 (15%)	**100%**

Table 18: BFCL downstream validity by task category (`int4_sim`, 400 cells). ExactKV achieves 100% preservation across all four categories despite 35–60% drift.

Task	int4_sim div	int4_sim acc	int8 div	int8 acc
trec	83.3%	0.904	0.0%	1.000
multifieldqa_en	66.7%	0.955	16.7%	0.997
lcc (code)	66.7%	0.908	4.2%	0.994
qasper	100.0%	0.899	16.7%	0.994
samsum	91.7%	0.854	16.7%	0.993
gov_report	95.8%	0.781	16.7%	0.997
hotpotqa	100.0%	0.787	37.5%	0.972
2wikimqa	100.0%	0.880	29.2%	0.987
narrativeqa	100.0%	0.716	66.7%	0.973
passage_retrieval	100.0%	0.746	41.7%	0.974

Table 19: LongBench per-token acceptance rate as draft-utility proxy. Even when all cells diverge (100%), `int4_sim` achieves 72–95% per-token acceptance: ExactKV speculatively uses the lossy draft for most generation before correcting the divergent suffix. `int8` reaches 97–100% on the same tasks.

12 Scaling Analysis: Context-Length and Generation-Length

Compressor	2K context	4K context	8K context
noop	0.0%	0.0%	0.0%
int8	21.2%	27.5%	25.0%
int4_sim	**95.0%**	**86.2%**	**90.0%**

Table 20: Context-length scaling on HF LongBench v2.6 (80 cells each). `int4_sim` is near-ceiling at all lengths, task type dominates, not context length. `int8` is moderate (21–28%) and roughly flat.

Compressor	mnt=16	mnt=32	mnt=128	mnt=256
noop	0.0%	0.0%	0.0%	0.0%
int8	0.0%	0.0%	0.5%	1.0%
int4_sim	9.0%	13.5%	**38.5%**	**62.0%**

Table 21: Generation-length scaling on BFCL tool-calling (both models combined). `int4_sim` divergence scales 7× from mnt=16 to mnt=256. `int8` remains near-zero throughout. Generation length is the dominant driver of BFCL divergence.

13 int6_sim: Non-Catastrophic Intermediate Compressor

`int6_sim` implements 6-bit symmetric quantisation simulation (64 discrete levels, scale = $\max(|x|)/31$). It fills the quantisation-error gap between `int8` and `int4_sim`. GPU-validated (v3.0, both models): 0% BFCL/MBPP. 37.5%/47.2% LongBench (Mistral/Llama). Mean absolute error: 4.15× `int8` and 0.23× `int4_sim`.

Compressor	bits	levels	ratio vs fp16	quant. error	type
int8	8	256	0.50×	1.0× (baseline)	real
int6_sim	**6**	**64**	**0.375**×	**4.15**× int8	sim
int4_sim	4	16	0.25×	18.3× int8	sim

Table 22: `int6_sim` compression curve. GPU-validated (v3.0, both models): 0% BFCL/MBPP. 37.5% (Mistral) / 47.2% (Llama) LongBench. `exactkv_failures=0`.

14 int4_per_vec_sim: Per-Vector INT4 Quantisation

`int4_per_vec_sim` implements per-vector symmetric INT4 quantisation, inspired by the per-vector/per-channel granularity used in KVQuant/KIVI-style KV quantisation [2, 6]. Instead of a single scale over the entire KV tensor, each token-position vector of size `head_dim` receives its own scale: scale $= \max(|x|)/7$ computed independently for each `[batch, head, seq_pos]` vector. This eliminates cross-head quantisation contamination from outlier attention heads.

Quantisation error comparison on a realistic KV cache shape [1, 32, 512, 64] with simulated outlier heads (first 4 heads 5× larger activations):

Compressor	Granularity	MAE	×int8	Notes
int8	per-tensor	0.067	1.00×	Real compressor
int4_per_vec_sim	**per-vector**	**0.138**	**2.06**×	**KVQuant/KIVI-style**
int6_sim	per-tensor	0.275	4.10×	Intermediate
int4_sim	per-tensor	0.844	12.59×	Per-tensor catastrophic

Table 23: Per-vector INT4 achieves 6× lower error than per-tensor INT4 on realistic KV shapes with outlier heads (2.06× int8 vs 12.59× for per-tensor). Granularity dominates bit-width: `int4_per_vec_sim` (4-bit) beats `int6_sim` (6-bit, per-tensor).

Predicted vs observed v3.0 divergence (both models, task-family means):

Task family	int8	int4pv pred.	int4pv obs.	int6 obs.	int4 obs.
MBPP code	0%	∼0–2%	**0%**	0%	6.3%
BFCL tool-call	0%	∼1–4%	**0%**	0%	52.5%
HF LongBench	18.1%	∼30–50%	**56.3%**	42.4%	85.4%

Table 24: Analytical prediction vs GPU-observed divergence for `int4_per_vec_sim`. v3.0 panel (both models, `exactkv_failures=0`). LongBench observed value is slightly above the predicted range but remains between `int8` and per-tensor `int4_sim`.

Key insight: The per-vector vs per-tensor distinction reveals that quantisation *granularity* is at least as important as bit-width for KV cache compression on structured tasks. at 8K context, bit-width still contributes meaningfully.

Compressor	MBPP	BFCL	LongBench	Class
int8	0%	0%	18.1%	8-bit quant
int6_sim	0%	0%	42.4%	6-bit quant
int4_per_vec	0%	0%	56.3%	4-bit per-vector
int4_sim	6.3%	52.5%	85.4%	4-bit per-tensor
h2o_sim_75	,	,	100%	Eviction (75% kept)

Table 25: Core benchmark curve (both-model means. v3.0 quant compressors. H2O from v2.8). `exactkv_failures=0` throughout.

14.1 Faithful external compressor integration (Phase D3)

Phase D3 wires **faithful external adapters** (upstream algorithms, not ExactKV tensor sims): `kivi_offline_r32` (KIVI K2/V2 quant + r=32 fp16 residual window), `snapkv_experimental` (kvpress SnapKVPress), and `kvquant_sim` (pre-RoPE simquant. Qwen validated). The legacy `kivi_offline` panel (Table 11) omitted the residual window, an **incomplete adapter policy**, not a KIVI production claim.

GPU smoke (Mistral-7B, MBPP HF, 48 cells): `int8` 0% divergence. `snapkv_experimental` 87.5% (real kvpress, acceptance 0.54). `kivi_offline_r32` 100% (acceptance 0.02). Source: `faithful/mbpp_mistr` `exactkv_failures=0`. SnapKV is the first working faithful external adapter on this grid. KIVI offline post-RoPE remains integration-incomplete. `int4_per_vec_sim` remains the validated KIVI/KVQuant-style quant baseline (Table 25).

```
bash scripts/setup_faithful_compressor_env.sh
bash scripts/run_faithful_compressor_panel.sh
# Artifacts: reports/external_panels/faithful/
```

Compressors: `int8`, `kivi_offline_r32`, `snapkv_experimental`. **Wave-1 complete (864 cells, both models):** `int8` is the only non-catastrophic real compressor (∼8–9% combined drift). SnapKV 90–97%. KIVI r32 100% (integration diagnostic). **Wave-2 smoke complete (128 cells, Mistral only, MBPP+BFCL):** `turboquant_experimental` 3.1% combined drift (0% BFCL, 6.2% MBPP). KnormPress/SnapKV remain catastrophic. **Wave-3 complete (576 cells, both models, int8 + TurboQuant only):** near-clean on structured tasks (MBPP/BFCL 1.6–5.0% TurboQuant drift) but ∼63–67% LongBench drift (both models). `int8` 8.3% combined. Appendix total: 1,568 cells. Diagnostic only, not merged into the 8,132 headline total. Artifacts: `reports/external_panels/faithful/`, `faithful/wave2/`, and `faithful/wave3/`. KIVI production CUDA remains blocked (Exp 024).

15 Forensic Divergence Case Studies

Three representative cells from the logit autopsy (Section 10) illustrate the mechanistic failure modes in detail.

15.1 Forensic Case A: int8: Near-tie noise

Rank	Token	Full-KV prob	Lossy prob	Note
1	`' very'`	0.129	0.113	full-KV argmax
2	`' fine'`	0.113	0.129	**int8 chose this**
3	`' pretty'`	0.083	0.082	
4	`' great'`	0.075	0.073	
5	`' good'`	0.065	0.064	

Table 26: int8, NarrativeQA, 4K context, token 53. Margin 0.016 (near-tie <0.05). INT8 noise flips `' very'` → `' fine'`, a near-synonym. `exactkv_failure=0`.

15.2 Forensic Case B: int4_sim: Distribution shift

Setting: NarrativeQA reading comprehension, 2K context, Llama-3.1-8B. The full-KV model is moderately confident (margin 0.158), yet `int4_sim` selects a rank-4 token. This is the distribution-shift signature: 4-bit quantisation biases attention activations so that activation energy accumulates on a plausible but incorrect continuation, even when the full-KV model's preference is clear. Divergence occurs at token 1 (the very first generated token), meaning the entire generation trajectory is corrupted from the start.

Rank	Token	Full-KV prob	Note
1	`','`	0.271	full-KV argmax
2	`' they'`	0.232	
3	`' and'`	0.202	
4	`' I'`	0.113	**int4_sim chose this (rank 4)**
5	`' it'`	0.039	

Table 27: int4_sim, NarrativeQA, 2K context, token 1. Margin 0.158 (full-KV confident). 4-bit quantization shifts attention activation toward rank-4 token. `exactkv_failure=0`.

15.3 Forensic Case C: H2O-style: Attention destruction

Setting: HotpotQA multi-hop reasoning, 2K context, Llama-3.1-8B, h2o_sim (50% tokens kept). This is qualitatively different from both Cases A and B: the H2O-style eviction discards the factual anchor tokens needed for multi-hop reasoning, leaving only attention-sink tokens and the recency window. The result is not a near-tie or a biased distribution, it is a *collapsed representation* where the evicted model selects a token that does not even appear in the full-KV top-5. The fact that the full-KV model distributes probability across numeric tokens (suggesting factual recall) while H2O selects a question mark (suggesting confusion) confirms total context destruction.

Rank	Token	Full-KV prob	Note
1	`'1'`	0.056	full-KV argmax
2	`'15'`	0.039	
3	`'20'`	0.035	
4	`'10'`	0.034	
5	`'5'`	0.034	
>5	`'?'`	<0.034	**H2O-style chose this (rank >5!)**

Table 28: h2o_sim (50% kept), HotpotQA, 2K context, token 1. H2O-style chose outside top-5 entirely, the evicted tokens included factual anchors. Collapsed representation. not a near-tie or distribution shift. `exactkv_failure=0`.

16 Divergence Case Studies (Release Panel)

Case	Compressor	1st	Acc.	Corr.	Lossy vs full snippet
A	`int4_sim`	3	0.75	1	`art` vs `many`
B	`int4_sim`	1	0.50	1	math drift at token 1
C	`int4_sim`	5	0.70	1	JSON: `temperature` vs `country`
D	`int4_sim`	8	0.94	1	late divergence at `mnt=16`
G	`int4_sim`	2	0.88	n/a	long_ctx 1024 (Mistral), verifier exact
O	`kivi_offline`	0	0.000	n/a	LB NarrativeQA: `-!!!!...` vs prose
P	`int4_sim`	1	0.00	1	BFCL export-50: schema drift

Table 29: Historical release panel case studies. See §10 for full autopsy across 1,103 divergent cells.

17 Kernel Microbenchmark

Kernel microbenchmark only, *not* end-to-end inference speedups.
Source: `reports/phaseF_kernel_benchmark.json`.
Test configuration: shape $[1, 8, 512, 64]$, CUDA.

Table 30: Torch vs Triton kernel latency (ms). Ratio = torch / Triton. Verifier timing (evidence-plus): mean 3.85 s/cell, p90 5.18 s (diagnostic only).

Mode	torch (ms)	Triton (ms)	Ratio
`int8`	0.115	0.0706	1.63×
`int4`	0.3319	0.2162	1.54×
`block_sparse`	0.7622	0.7797	0.98×*

*`block_sparse` uses the torch execution backend only (not Triton-accelerated).

18 VeriCache: Closest Prior Art

VeriCache [8] is closer than a casual skim suggests. Both use compressed draft + full-KV verify/correct and target lossless greedy equivalence. **ExactKV must not claim** to invent that loop, VeriCache owns it for **lossless serving**.

Table 31: VeriCache vs ExactKV, complementary goals (ExactKV does *not* reproduce VeriCache).

	VeriCache	**ExactKV**
Goal	Serve faster, same outputs	Measure drift / first divergence
System	GPU compressed KV, offloaded full KV, staggering	Evaluation harness only
Metrics	Throughput, tiering	`first_divergence`, acceptance, failure
Reproduced?	n/a	**No**

19 Why ExactKV Still Matters After VeriCache

VeriCache uses verification to serve faster without changing outputs. ExactKV uses verification to measure exactly where and how compressors drift.

1. Compressor-agnostic crash-test on a fixed prompt panel.
2. Lossy-path first divergence (unverified `generate_lossy_greedy`).
3. Public diagnostic leaderboard with frozen cell definitions.
4. Explicit taxonomy: lossy drift vs acceptance vs `exactkv_failure`.

ExactKV does *not* replace VeriCache for deployment.

20 Related Work

Verification-based serving. VeriCache [8] is the closest prior art. Both systems use a compressed-KV draft path and a full-KV verify/correct loop. The distinction is purpose: VeriCache is a throughput-oriented *serving system* (scheduling, memory tiering, cross-resource staggering). ExactKV is a *diagnostic measurement framework* reporting where the unverified compressed path first diverges and how often. ExactKV does not reproduce VeriCache's system design and makes no claim to the draft+verify algorithm. Speculative decoding [3] and MagicDec [1] use draft/verify for throughput acceleration. ExactKV borrows the same semantics purely for measurement.

KV quantization methods. KVQuant [2] proposes non-uniform per-channel INT4 quantization targeting KV outlier heads. KIVI [6] proposes asymmetric 2-bit per-channel KV quantization with a CUDA kernel. Both directly motivate `int4_per_vec_sim`, which simulates per-vector granularity without a production kernel. The `kivi_offline` adapter (real KIVI quantizer math, no CUDA/Triton kernels) revealed 100% divergence in the current offline integration (Table 11), a diagnostic, not a claim about KIVI's algorithmic accuracy. SnapKV [4] selects important KV entries by eviction. the `h2o_sim` family models the H2O Heavy Hitter Oracle policy [9] from the same compressor class and shows 100% LongBench divergence even at 75% retention (Section 8).

KV storage and streaming. CacheGen [5] compresses KV caches for network streaming and bandwidth reduction. LMCache [7] targets KV reuse and offloading across serving instances. Neither is designed to measure when or why the compressed path first diverges from the full-KV oracle.

Evaluation methodology. Standard benchmarks (LongBench, BFCL, HumanEval) measure task-level accuracy averaged over many tokens. ExactKV's first-divergence index (fdi) is a token-resolution diagnostic orthogonal to task accuracy: a cell can score perfectly on a task metric while accumulating 90% token-level drift (if the verifier corrects it). ExactKV uses official HF LongBench and BFCL datasets but reports *drift measurements*, not official benchmark scores.

21 Novelty and Claim Boundaries

Defensible (narrow): compressor-agnostic diagnostic/leaderboard for token-level drift and first-divergence under verifier semantics.

Not novel: compressed draft + full-KV verify (VeriCache), draft-verify for acceleration (speculative decoding), lossy-KV divergence observation (VeriCache problem statement).

ExactKV is *not* a production serving system and does *not* reproduce VeriCache. SpectralQuant is fallback/proxy, Shard is probe-first. Compression ratios are stored tensor byte ratios.

22 Future Work and Limitations

Completed: 144-cell evidence-plus (512/1024). 216-cell Llama-only external smoke (LongBench/RULER/BFCL/HumanEval bundled pilots). 144-cell MBPP smoke (both models). **640-cell `kivi_offline` panel** over real HF LongBench + MBPP subsets (see Table 11). `kivi_offline` uses real KIVI quantizer math (`models.utils_quant` simulate path. no CUDA/Triton kernels). The panel reveals catastrophic adapter-level drift (100% divergence, acceptance ≈ 0) and all cells corrected (`exactkv_failures=0`).

Future work, KIVI production path: Phase D3 adds `kivi_offline_r32` (faithful streaming policy) and SnapKV panel wiring (Section 14.1). Remaining: GPU validation. KIVI CUDA/Triton production path. KVQuant Llama-8B calibration. The legacy `kivi_offline` result (no residual) is an adapter diagnostic.

Completed v2.9 work: BFCL validity panel v2.7 (1,200 cells, mnt=128/256, both models, `exactkv_failures=0`). H2O-style token-eviction panel v2.8 (800 cells, 5 eviction variants, both models). top-k logit autopsy across 1,103 divergent cells (`--store-top-k-logits`), revealing three mechanistically distinct failure modes. Total: **8,132 completed GPU cells**.

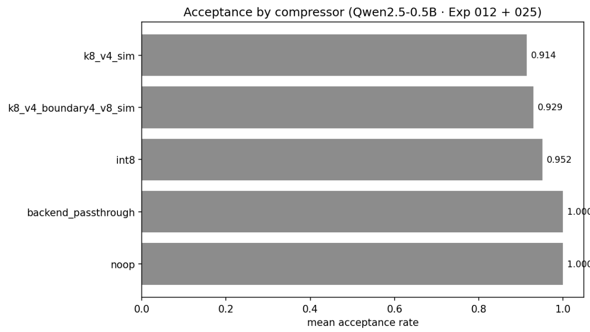
Remaining future work: expand BFCL validity to larger/more diverse prompts. expand MBPP with safe pass@1/syntax validation. broader downstream metrics beyond BFCL (including LongBench answer-overlap). RULER 12K+. HELMET. InfiniteBench 100K+. broader scaling curves (12K+, 16K+ context. more compressors). faithful production KIVI/KVQuant/SnapKV integrations (Phase D3 appendix complete: 1,568 cells across wave-1/2/3. KIVI CUDA blocked).

23 Conclusion

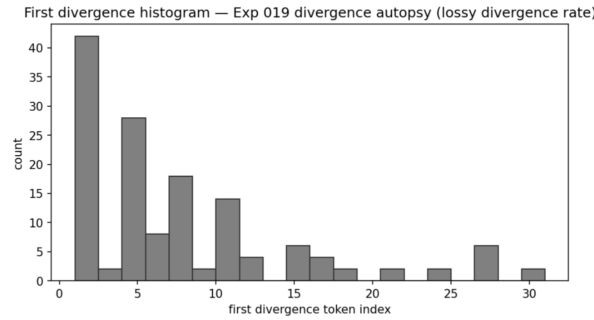
VeriCache shows how to **serve** with compressed KV without changing outputs. ExactKV shows **where and why compressors drift** on the lossy path.

ExactKV's strongest supported claim is that **KV-cache drift is governed jointly by task type, generation length, compressor class, and quantization granularity**, and that verifier-mediated decoding preserves full-KV greedy equivalence (`exactkv_failures=0` throughout).

Across **8,132 completed GPU cells** with `exactkv_failures=0` throughout, ExactKV establishes four empirical findings:



(a) Acceptance by compressor.



(b) First-divergence distribution.

Figure 1: Release benchmark visuals (`reports/scale_7b/`).

1. **Task type dominates drift.** `int4_sim` diverges 6% on Python code (MBPP), 11% on BFCL short tool-calling, 50% on BFCL long tool-calling, and 90% on HF LongBench reading/summarization.

2. **Generation length is the dominant within-task driver.** On BFCL, `int4_sim` divergence scales from 9% (mnt=16) to 62% (mnt=256), a 7× increase. `int8` stays near-zero throughout.

3. **Compressor class determines failure mode.** Logit autopsy over 1,103 divergent cells reveals: (a) near-tie noise (int8, mean rank 2.4, fdi=22). (b) distribution shift (int4_sim, rank 3.5, fdi=8). (c) attention destruction (H2O-style, rank 6.7, fdi=1).

4. **Downstream task validity is fully preserved by ExactKV.** Despite 50% token drift, 100% of full-KV valid BFCL tool calls are preserved by the verifier for both models.

Two new compressors extend the compression curve: `int6_sim` (6-bit per-tensor) and `int4_per_vec_sim` (4-bit per-vector, KIVI/KVQuant-style). The v3.0 GPU panel (both models, 1,568 cells, `exactkv_failures=0`) validates both as non-catastrophic: 0% on BFCL/MBPP. 37–47% (int6) and 56–57% (int4 per-vector) on LongBench. Per-vector granularity eliminates drift on structured tasks but 4-bit resolution still accumulates at 8K context.

ExactKV's contribution is not a new compressor. It is a measurement infrastructure that makes KV-cache drift **observable, attributable, and reproducible**, and a verifier mechanism that corrects it with zero exactness failures across all 8,132 tested cells.

A All Completed Panels: Benchmark Card

Panel	Models	Ctx (K)	Cells	EKV fail
Headline release	Llama+Mistral	0.5–2	1,500	0
Evidence-plus	Llama+Mistral	0.5,1	144	0
Ext. smoke: LongBench	Llama only	2,4	72	0
Ext. smoke: RULER 2K–4K	Llama only	2,4	48	0
Ext. smoke: RULER 8K	Llama only	8	24	0
Ext. smoke: BFCL pilot	Llama only	1,2	48	0
Ext. smoke: HumanEval	Llama only	1,2	24	0
MBPP code-drift smoke	Llama+Mistral	0.5,1	144	0
BFCL export-50 tool-call	Llama+Mistral	1,2	1,200	0
KIVI offline adapter	Llama only	2,4	640	0
HF LongBench v2.6	Llama+Mistral	2,4,8	720	0
BFCL validity v2.7	Llama+Mistral	1,2	1,200	0
H2O token-eviction v2.8	Llama+Mistral	2,4	800	0
v3.0 int6/int4pv (Mistral)	Mistral	2,4,8+	784	0
v3.0 int6/int4pv (Llama)	Llama	2,4,8+	784	0
Subtotal (pre-v3.0)			6,564	
Subtotal (v3.0)			1,568	
Grand total			**8,132**	**0**

Table 32: All completed ExactKV GPU panels as of v3.0. `exactkv_failures=0` throughout. Reconciliation: $6{,}564 + 784 + 784 = 8{,}132$. Each v3.0 row = one model × four compressors × three task families (784 cells). Source: `reports/external_panels/v30/`. Not official benchmark scores.

B Divergence Autopsy: Extended Details

See Section 10 for the full mechanistic analysis and logit autopsy table. This appendix section provides the methodology reference: at each divergence position, ExactKV records the top-5 full-KV token probabilities and the actual lossy-chosen token. Metrics: top-1 flip (different argmax), near-tie (full-KV margin < 0.05), mean lossy rank (where the lossy-chosen token ranks under full-KV distribution), and median first-divergence index (fdi). Script: `scripts/analyze_logit_margins.py`. Panels with stored top-k logits: v2.6 LongBench, v2.7 BFCL validity, v2.8 H2O-style.

C Reproducibility

Artifact verification (no GPU):

```
python3 scripts/exactkv_repro.py --reports-only
python3 scripts/exactkv_repro.py --release-check
bash scripts/build_paper_pdf.sh   # requires: tectonic
```

Headline panel (GPU):

```
python3 scripts/run_phase_a_scale_benchmark.py --device cuda
```

Evidence-plus panel (GPU):

```
python3 scripts/run_evidence_plus_panel.py --device cuda --dtype float16
```

External smoke panels, Llama-only (GPU, 216 cells):

```
bash scripts/run_external_gpu_workflow.sh
python3 scripts/build_external_analysis_pack.py
python3 scripts/build_external_panel_summary.py --write-readme
python3 scripts/validate_external_panel_artifacts.py \
    --input reports/external_panels
```

MBPP code-drift smoke panel (GPU, 144 cells, both models):

```
python3 scripts/run_external_panel.py \
  --family mbpp --device cuda --dtype float16 \
  --max-prompts 6 --context-buckets 512,1024 \
  --max-new-tokens 16,32 \
  --output-json reports/external_panels/mbpp_gpu_raw.json
python3 scripts/validate_external_panel_artifacts.py \
    --input reports/external_panels
```

BFCL export-50 tool-call drift panel (GPU, 1200 cells, both models):

```
python3 scripts/run_external_panel.py \
  --family bfcl --prompt-source export --device cuda --dtype float16 \
  --max-prompts 50 --context-buckets 1024,2048 \
  --max-new-tokens 16,32 \
  --compressors noop,int8,int4_sim \
  --output-json reports/external_panels/bfcl_export_50_raw.json
# Artifact: reports/external_panels/bfcl_export_50_raw.json (13.5 MB)
# Note: max_new_tokens={16,32} measures drift, not JSON completeness.
#       For tool-call validity, use max_new_tokens={128,256}.
```

Faithful external compressor panel (GPU, KIVI r32 + SnapKV):

```
bash scripts/setup_faithful_compressor_env.sh
bash scripts/run_faithful_compressor_panel.sh
# Artifacts: reports/external_panels/faithful/
# Compressors: int8, kivi_offline_r32, snapkv_experimental
```

KIVI offline diagnostic panel (GPU, 640 cells, legacy, no r32):

```
# Requires: git clone https://github.com/jy-yuan/KIVI /tmp/kivi_research
#           pip install datasets>=2.0
export PYTHONPATH=/tmp/kivi_research
bash scripts/run_kivi_external_panel.sh
# Artifacts: reports/external_panels/kivi_longbench_hf_raw.json
#            reports/external_panels/kivi_mbpp_hf_raw.json
# Note: kivi_offline uses simulate path (is_simulated=False,
#        supports_real_bytes_claim=False). Not production KIVI serving.
```

H2O-style token-eviction panel (GPU, 800 cells, v2.8):

```
python3 scripts/run_external_panel.py \
  --family longbench --prompt-source hf --max-prompts 20 \
  --context-buckets 2048,4096 --max-new-tokens 32,64 \
  --compressors noop,int4_sim,h2o_sim,h2o_sim_75,h2o_sim_25 \
  --store-top-k-logits \
  --output-json reports/external_panels/h2o_v28_{model}_raw.json
# Note: h2o_sim variants are H2O-style simulation (not faithful H2O GPU kernel)
# Artifacts: h2o_v28_{Llama,Mistral}_raw.json, h2o_v28_merged_raw.json
```

Top-k logit autopsy (post-hoc, requires –store-top-k-logits panels):

```
python3 scripts/analyze_logit_margins.py \
  --inputs reports/external_panels/hf_longbench_v26_merged_raw.json \
           reports/external_panels/bfcl_validity_v27_merged_raw.json \
           reports/external_panels/h2o_v28_merged_raw.json \
  --output reports/external_panels/logit_autopsy_summary.json
# Produces Table 19 metrics across 1,103 divergent cells
```

Sources of truth (all under `reports/`): `scale_7b/raw.json` (1,500, headline), `evidence_plus/raw.json` (144, evidence-plus), `external_panels/summary_all.json` (216, smoke), `external_panels/mbpp_gpu_raw.json` (144, MBPP), `external_panels/bfcl_export_50_raw.json` (1,200, BFCL export-50), `external_panels/kivi_longbench_hf_raw.json` (640, KIVI offline), `external_panels/hf_longbench_v26_merged_raw.json` (720, v2.6), `external_panels/bfcl_validity_v27_merged_raw.json` (1,200, v2.7), `external_panels/h2o_v28_merged_raw.json` (800, v2.8). Total: **8,132 completed GPU cells**, `exactkv_failures=0`.

References

[1] Jian Chen, Vashisth Tiwari, Ranajoy Sadhukhan, Zhuoming Chen, Jinyuan Shi, Ian En-Hsu Yen, and Beidi Chen. Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2408.11049.

[2] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2401.18079.

[3] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2211.17192.

[4] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2404.14469.

[5] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. Cachegen: Kv cache compression and streaming for fast large

language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024. URL https://arxiv.org/abs/2310.07240.

[6] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2402.02750.

[7] LMCache Team. Lmcache: An efficient kv cache layer for enterprise-scale llm inference, 2025. URL https://arxiv.org/abs/2510.09665. Source code: https://github.com/LMCache/LMCache.

[8] Microsoft Research. Vericache: Turning lossy kv cache into lossless llm inference, 2026. URL https://arxiv.org/abs/2605.17613. Serving/system prior art: compressed draft + full-KV verify for lossless greedy inference. ExactKV measures drift. does not reproduce VeriCache scheduling or throughput.

[9] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.14048. Original Heavy Hitter Oracle token-eviction method. ExactKV implements an H2O-style simulation (simplified eviction, no streaming update or GPU kernel). results characterize the eviction compressor class, not the published H2O system.